

Sommaire

1. Contexte du projet

1.1. Présentation du projet

1.2. Date de rendu du projet

2. Besoins fonctionnels

3. Ressources nécessaires à la réalisation du projet

3.1. Ressources matérielles

3.2. Ressources logicielles

4. Gestion du projet

5. Conception du projet

5.1. Le front-end

5.1.1. Wireframes

5.1.2. Maquettes

5.1.3. Arborescences

5.2. Le back-end

5.2.1. Diagramme de cas d'utilisation

5.2.2. Diagramme d'activités

5.2.3. Modèles Conceptuel de Données (MCD)

5.2.4. Modèle Logique de Données (MLD)

5.2.5. Modèle Physique de Données (MPD)

6. Technologies utilisées

6.1. Langages de développement Web

6.2. Base de données

7. Sécurité

7.1. Login et protection des pages administrateurs

7.2. Hachages des mots de passe avec Bcrypt

7.3. Protection contre les attaques XSS (Cross-Site Scripting)

7.4. Protection contre les injections SQL

1. Contexte du projet

1.1. Présentation du projet

Votre agence web a été sélectionnée par le comité d'organisation des jeux olympiques de Los Angeles 2028 pour développer une application web permettant aux organisateurs, aux médias et aux spectateurs de consulter des informations sur les sports, les calendriers des épreuves et les résultats des JO 2028.

Votre équipe et vous-même avez pour mission de proposer une solution qui répondra à la demande du client.

1.2. Date de rendu du projet

Le projet doit être rendu au plus tard le 18/12/2025.

2. Besoins fonctionnels

Le site web devra avoir une partie accessible au public et une partie privée permettant de gérer les données.

Les données seront stockées dans une base de données relationnelle pour faciliter la gestion et la mise à jour des informations. Ces données peuvent être gérées directement via le site web à travers un espace administrateur.

3. Ressources nécessaires à la réalisation du projet

REPRENDRE RÉPONSES MISSION 1

3.1. Ressources matérielles

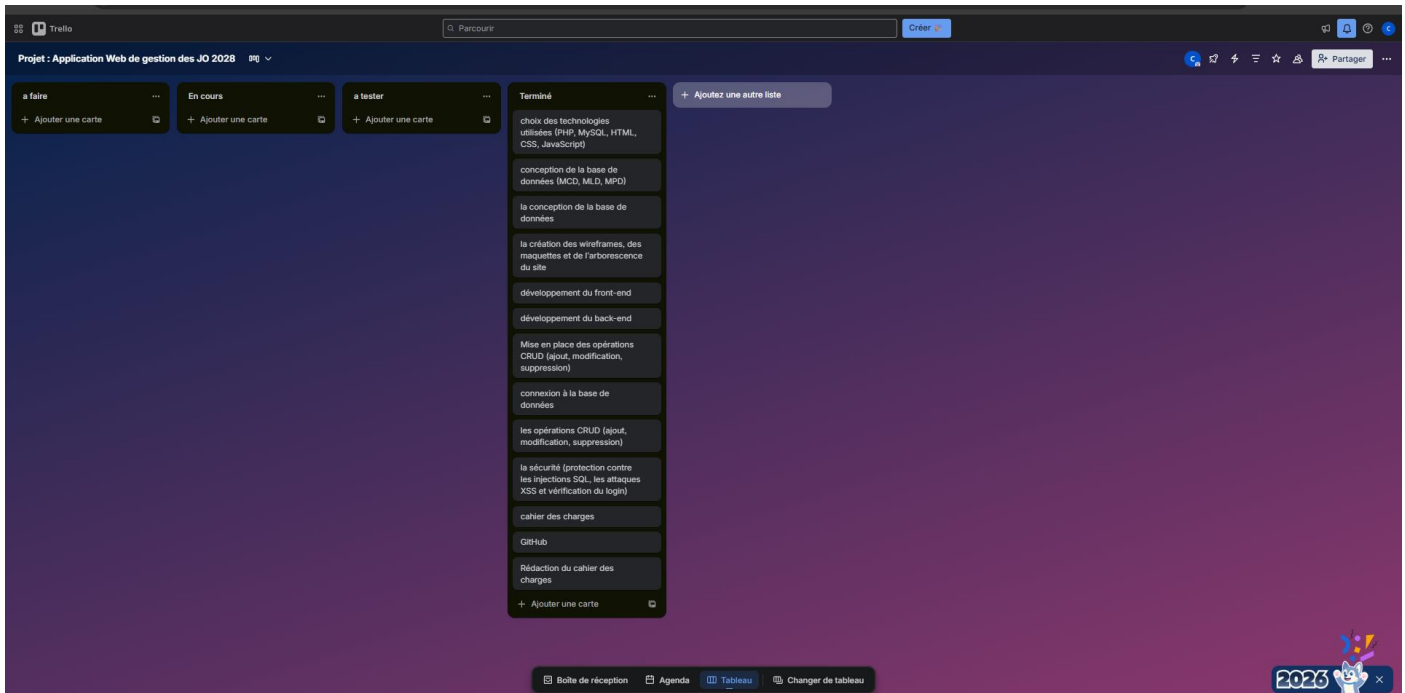
- PC
- Connexion internet
- Serveur local
- Périphérique de travail

3.2. Ressources logicielles

- Visual studio code
- Php
- Mysql
- Mamp
- Mocodo
- Java-script
- Html-CSS

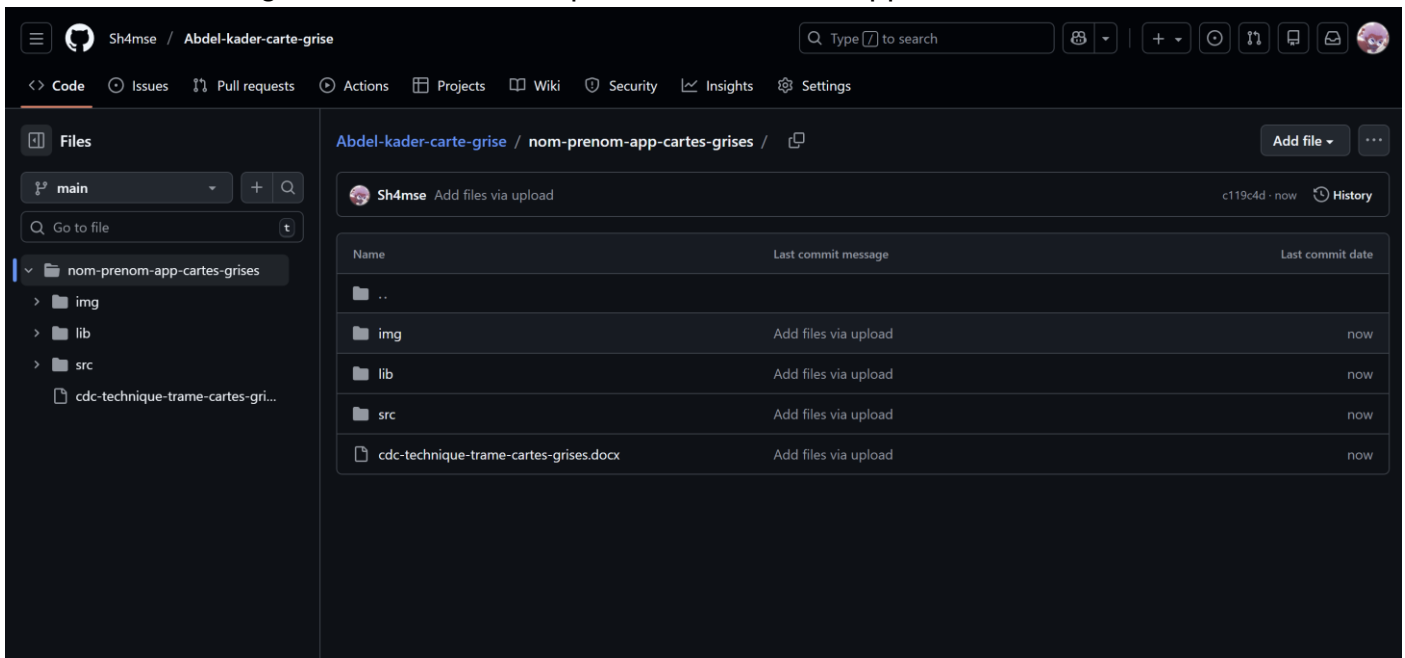
4. Gestion du projet

Pour réaliser le projet, nous utiliserons la méthode Agile Kanban. Nous utiliserons également l'outil de gestion de projet en ligne Trello.



INSÉREZ UNE CAPTURE D'ÉCRAN DE VOTRE TRELLO

Nous travaillons également sur GitHub, plateforme de développement collaboratif.



5. Conception du projet

5.1. Le front-end

REPRENDRE RÉPONSES MISSION 4

5.1.1. Wireframes



Propriétaire	Véhicule	Date début	Date fin	Modifier	Supprimer
John Doe	AB-123-CD	01/01/2023	31/12/2023	Modifier	Supprimer
Jane Doe	EF-456-GH	01/01/2023		Modifier	Supprimer
Alice Smith	IJ-789-KL	01/06/2023		Modifier	Supprimer

Ajouter une possession Fermer

Propriétaire:

John Doe ▼

Véhicule:

AB-123-CD ▼

Date début (JJ/MM/AAAA):

Date fin (JJ/MM/AAAA):

Enregistrer

Propriétaire:

Alice Smith ▼

Véhicule:

EF-456-GH ▼

Date début (JJ/MM/AAAA):

16/12/2022

Date fin (JJ/MM/AAAA):

18/02/2023

Enregistrer

5.1.2. Maquettes



Liste des possessions					
Propriétaire	Véhicule	Date début	Date fin	Modifier	Supprimer
John Doe	AB-123-CD	01/01/2023	31/12/2023	Modifier	Supprimer
Jane Doe	AB-123-CD	24/12/2025	24/12/2030	Modifier	Supprimer
Jane Doe	EF-456-GH	01/01/2023		Modifier	Supprimer
Alice Smith	IJ-789-KL	01/06/2023		Modifier	Supprimer
Chattou Imad	IC-212-AK	24/12/2006		Modifier	Supprimer

Buttons: Ajouter une possession, Fermer

Formulaire possession

Propriétaire:

Véhicule:

Date début (JJ/MM/AAAA):

Date fin (JJ/MM/AAAA):

Formulaire possession

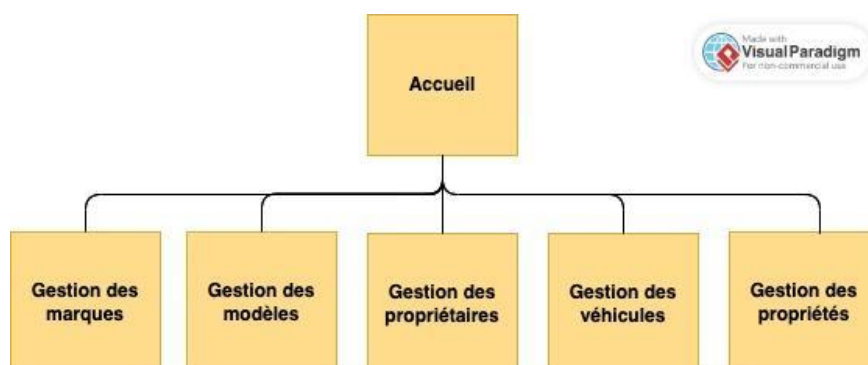
Propriétaire:

Véhicule:

Date début (JJ/MM/AAAA):

Date fin (JJ/MM/AAAA):

5.1.3. Arborescences



5.2. Le back-end

REPRENDRE RÉPONSES MISSIONS 2 ET 3

5.2.1. Diagramme de cas d'utilisation

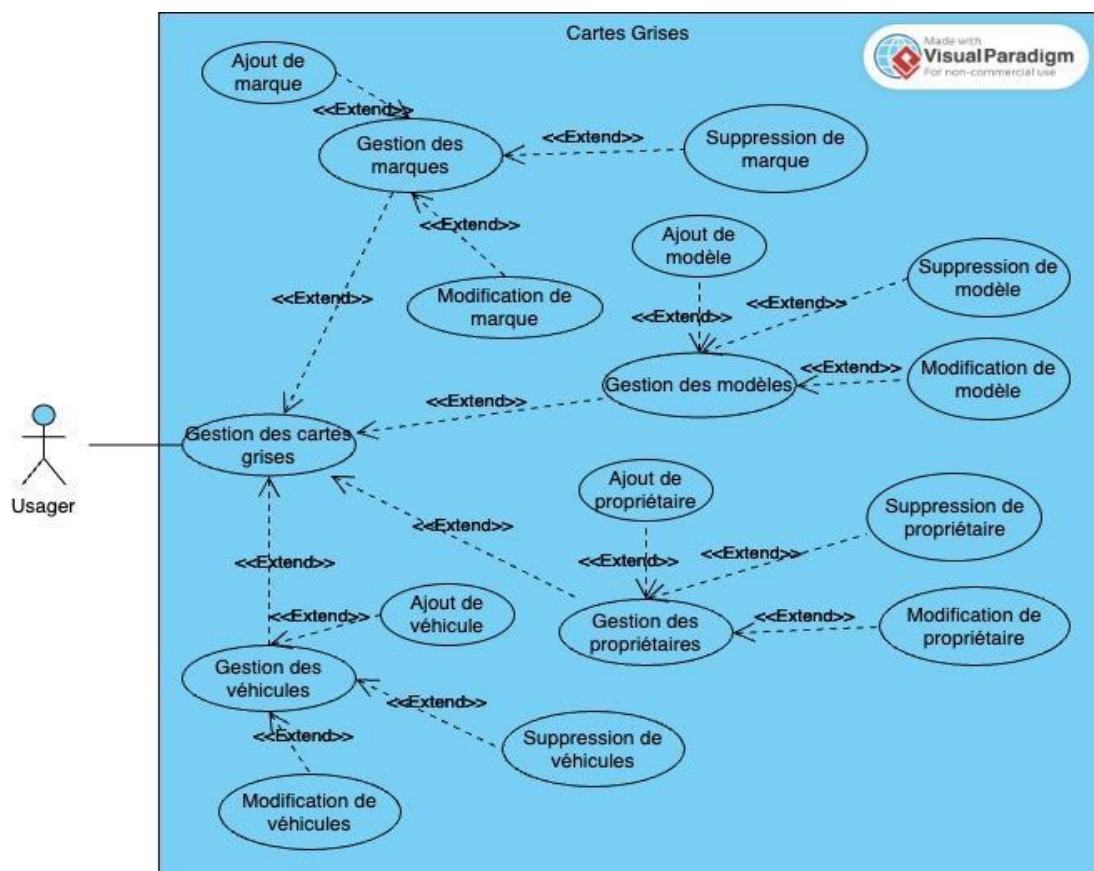
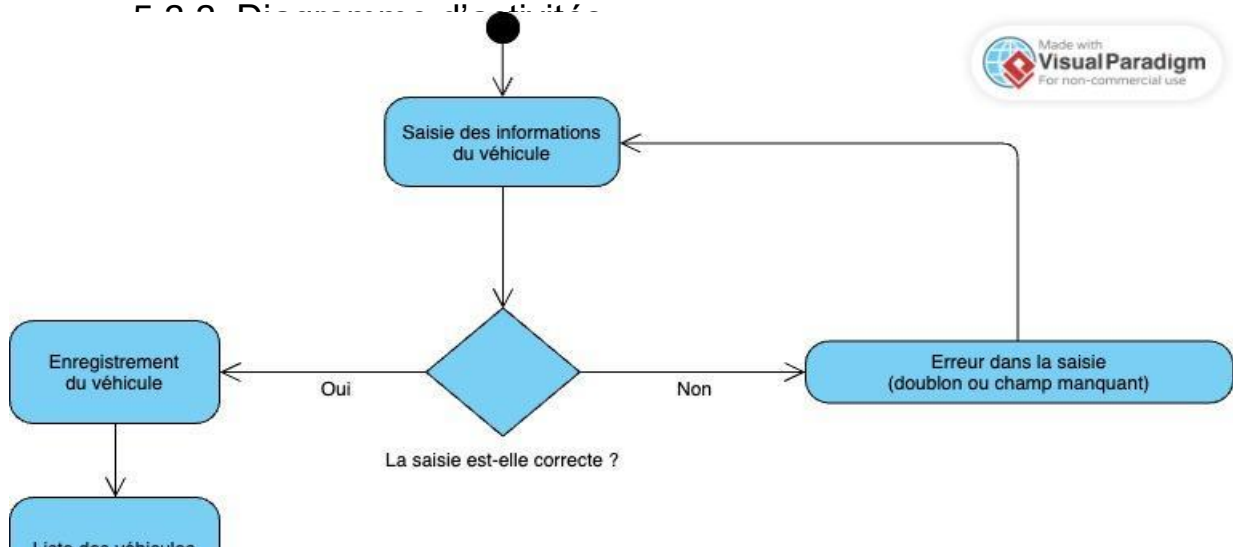
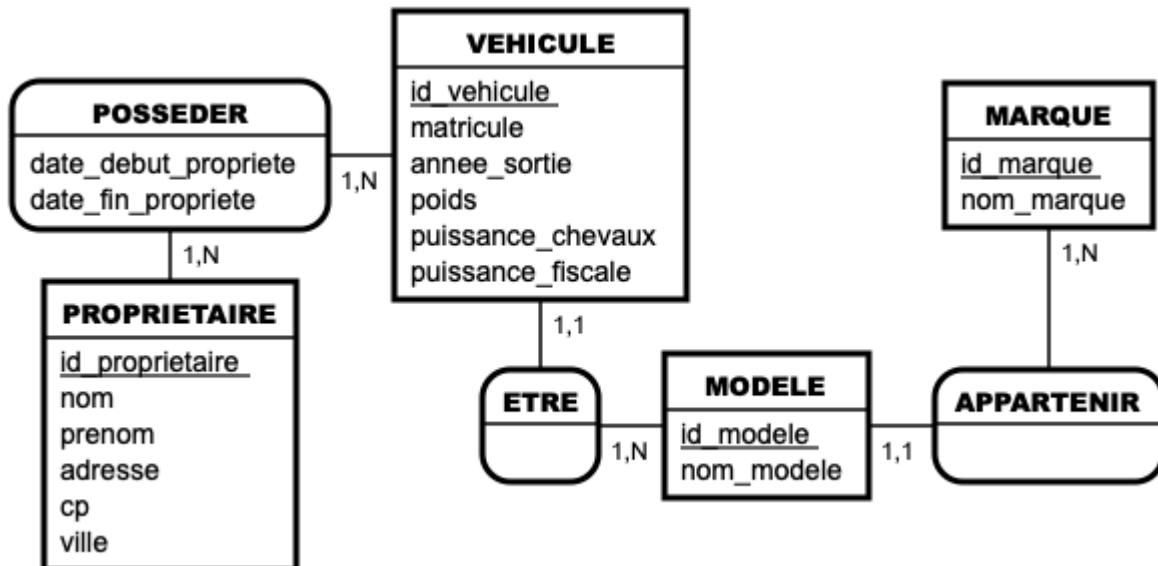


Figure 5.2.1. Diagramme de cas d'utilisation



5.2.3. Modèles Conceptuel de Données (MCD)

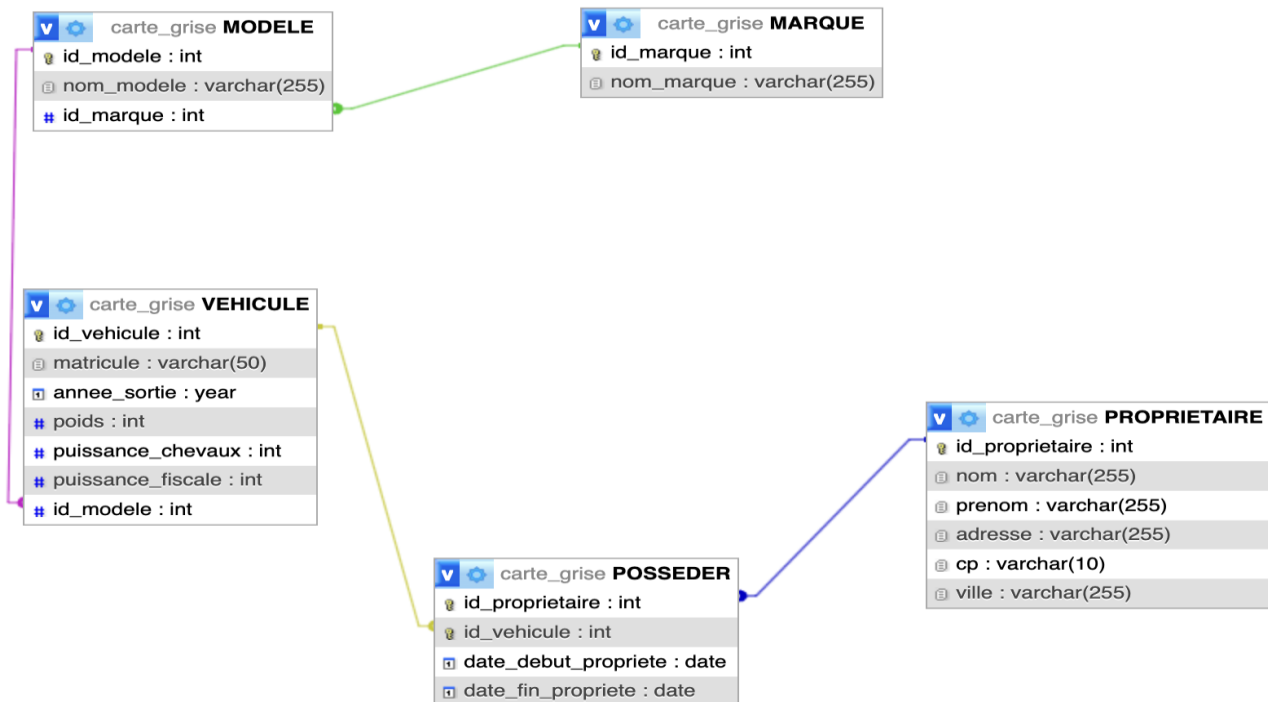


5.2.4. Modèle Logique de Données (MLD)

- MARQUE (id_marque, nom_marque)
Clé primaire : id_marque
- MODELE (id_modele, nom_modele, id_marque)
Clé primaire : id_modele
Clé étrangère : id_marque en référence à id_marque de MARQUE
- VEHICULE (id_vehicule, matricule, annee_sortie, poids, puissance_chevaux, puissance_fiscale, id_modele)
Clé primaire : id_vehicule
Clé étrangère : id_modele en référence à id_modele de MODELE

- PROPRIETAIRE (id_proprietaire, nom, prenom, adresse, cp, ville)
Clé primaire :id_proprietaire
- POSSEDER (id_proprietaire, id_vehicule, date_debut_propriete, date_fin_propriete)
Clé primaire :id_proprietaire, id_vehicule
Clés étrangères :id_proprietaire en référence à id_proprietaire de PROPRIETAIRE id_vehicule en référence à id_vehicule de VEHICULE

5.2.5. Modèle Physique de Données (MPD)



6. Technologies utilisées

REPRENDRE RÉPONSES MISSION 1

6.1. Langages de développement Web



6.2. Base de données



7. Sécurité

DÉFINISSEZ ET EXPLIQUEZ BRIÈVEMENT VOTRE SOLUTION.
AJOUTEZ SI VOUS LE SOUHAITEZ DES COURS EXTRAITS DE CODE.

7.1. LOGIN ADMIN (Titre conservé)

Définition de la solution :

L'accès à l'interface d'administration est protégé par une étape d'identification obligatoire. Il s'agit de s'assurer que l'utilisateur possède les droits requis avant de lui ouvrir les fonctionnalités de gestion.

- **Processus :** Saisie des identifiants via un formulaire, hachage et comparaison via BCrypt, puis initialisation d'une session utilisateur en cas de succès.

PAGES ADMIN

Il s'agit des espaces de travail réservés exclusivement aux gestionnaires du système. Ces interfaces permettent de piloter l'application (Tableau de bord, annuaire des utilisateurs, journaux d'événements et paramètres système).

PROTECTION

Mise en place de barrières de sécurité pour interdire l'usage des outils critiques par des personnes non autorisées.

Extraits de code

Java

```
// VALIDATION MANUELLE AVANT SUPPRESSION
int reponse = JOptionPane.showConfirmDialog(ProprietaireView.this,
    "Voulez-vous vraiment supprimer ce dossier ?",
    "Alerte de sécurité",
    JOptionPane.YES_NO_OPTION);

// CAPTURE DES ANOMALIES D'AFFICHAGE
try {
    new ProprietaireView(proprietaireController).showWindow();
} catch (Exception erreurAccès) {
    showError("Erreur d'interface Propriétaire", erreurAccès);
}
```

MESURES DE SÉCURITÉ :

- Double vérification auprès de l'utilisateur avant toute suppression.
- Interception des erreurs pour éviter le plantage de l'application.
- Isolation des méthodes de modification de données.

7.2. Hachages des mots de passe avec Bcrypt (Titre conservé)

Définition de la solution :

Bcrypt est une méthode de chiffrement unidirectionnelle spécifiquement créée pour le stockage des mots de passe. Au lieu d'enregistrer le texte en clair, on génère une empreinte numérique unique et irréversible. Sa force réside dans sa lenteur calculée, ce qui rend les attaques par dictionnaire ou force brute extrêmement coûteuses en temps pour un pirate.

ATOUTS DE BCRIPT :

- **Sel (Salt) automatique** : Protection native contre l'usage de tables de hachage pré-calculées (Rainbow Tables).
- **Évolutivité** : Possibilité de renforcer la sécurité en ajustant le coût de calcul selon la puissance des processeurs actuels.

Extraits de code

Java

```
// DÉTECTION DES ENREGISTREMENTS IDENTIQUES
public static boolean estDejaExistant(String numMatricule, Integer idAExclure) {
    String requêteSql = "SELECT COUNT(*) FROM VEHICULE WHERE matricule = ?";

    if (idAExclure != null) {
        requêteSql += " AND id_vehicule != ?";
    }

    try (Connection laConnexion = DBConnection.getConnection();
        PreparedStatement instruction = laConnexion.prepareStatement(requêteSql)) {

        instruction.setString(1, numMatricule);
        if (idAExclure != null) {
            instruction.setInt(2, idAExclure);
        }

        try (ResultSet resultat = instruction.executeQuery()) {
            if (resultat.next()) {
                return resultat.getInt(1) > 0;
            }
        }
    } catch (SQLException erreur) {
        System.err.println("Log Erreur (Unicité) : " + erreur.getMessage());
    }
    return false;
}
```

7.3. Protection contre les attaques XSS (Cross-Site Scripting) (Titre conservé)

Définition de la solution :

L'attaque XSS consiste à injecter des scripts malveillants dans une application. Si ces données ne sont pas nettoyées, le navigateur des autres utilisateurs exécutera ce code, permettant le vol de cookies ou la redirection vers des sites frauduleux.

FORMES D'ATTAQUES :

1. **XSS Persistant** : Le script est sauvegardé directement en base de données.
2. **XSS Non-persistant** : Le code transite temporairement par un paramètre d'URL ou un champ de recherche.

Extraits de code

Java

```
// NETTOYAGE DES ENTRÉES (Exemple Vehicule.java)
public static boolean verifPresence(String codePlate, Integer idIgnore) {
    String sqlCmd = "SELECT COUNT(*) FROM VEHICULE WHERE matricule = ?";
    // L'usage de setString() ci-dessous neutralise les tentatives d'injection
    // ... (Logique de connexion identique)
}

// RENDU SÉCURISÉ DANS L'INTERFACE (ProprietaireView.java)
```

```
// Ajout des lignes dans le composant JTable
vueTableau.addRow(new Object[]{
    p.getIdProprietaire(),
    p.getNom(),    // La JTable traite le texte comme du contenu brut
    p.getPrenom(), // Aucun script ne peut s'exécuter dans cette cellule
    p.getAdresse(),
    "Action : Supprimer"
});
```

STRATÉGIES APPLIQUÉES :

- **Neutralisation SQL** : Utilisation systématique de PreparedStatement.
- **Échappement automatique** : Utilisation de composants graphiques Java Swing qui ne traitent pas le HTML/JS.

7.4. Protection contre les injections SQL (Titre conservé)

Définition de la solution :

Une injection SQL survient lorsqu'un pirate parvient à glisser des commandes SQL à l'intérieur d'un formulaire. Sans protection, le serveur exécute ces ordres "parasites" comme s'ils faisaient partie du code légitime, risquant ainsi la fuite totale de la base de données ou sa destruction.

Extraits de code

Java

```
// USAGE DES REQUÊTES PARAMÉTRÉES
public static boolean enregistrerVehicule(String plaque, int anSortie, double masse,
                                          int cvVapeur, int cvFiscaux, int refModele) {

    // On utilise des points d'interrogation comme marqueurs de place (Placeholders)
    String commande = "INSERT INTO VEHICULE (matricule, annee_sortie, poids, " +
        "puissance_chevaux, puissance_fiscale, id_modele) " +
        "VALUES (?, ?, ?, ?, ?, ?)";

    try (Connection db = DBConnection.getConnection();
        PreparedStatement pstmt = db.prepareStatement(commande)) {

        // Attribution des valeurs avec typage rigoureux
        pstmt.setString(1, plaque);
        pstmt.setInt(2, anSortie);
        pstmt.setDouble(3, masse);
        pstmt.setInt(4, cvVapeur);
        pstmt.setInt(5, cvFiscaux);
        pstmt.setInt(6, refModele);

        return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        System.err.println("Echec de l'insertion : " + e.getMessage());
        return false;
    }
}
```

```
}  
}
```

POINTS CLÉS DU CODE :

- **Typage fort** : En utilisant `setInt()` ou `setString()`, le système vérifie que la donnée correspond au format attendu.
- **Transactions** : Utilisation de `commit` et `rollback` pour garantir que la base de données ne reste jamais dans un état instable en cas de bug.